

CS 1010: Introduction to Cryptology

Lecture Notes: Foundational Concepts & Classical Ciphers

Shibam Ghosh

September 3, 2026

Contents

1	Introduction to Cryptology	1
1.1	Why Cryptography?	2
2	Applications & Career Relevance	2
3	Classical Ciphers	2
3.1	The Caesar Cipher	3
3.2	The Shift Cipher (Caesar with Keys)	3
3.3	Monoalphabetic Substitution Cipher	3
3.4	Vigenère Cipher	4
4	Breaking the Vigenère Cipher using Kasiski's Method	5
4.1	Step 1: Estimating the Key Length	6
4.1.1	Estimating the Key Length Using IoC	6
4.1.2	Algorithm for Estimating the Key Length	6
4.2	Step 2: Recovering Each Key Letter Using Frequency Analysis	7
5	Symmetric and Asymmetric Key Cryptography	7
6	Symmetric Key Cryptosystem	8
6.1	Cipher Design Criteria	9

1 Introduction to Cryptology

At a foundational level, **cryptology** is the scientific study of protecting information in the presence of adversaries. It encompasses two primary, complementary branches:

Cryptography: Derived from the Greek words *kryptos* ("hidden") and *graphia* ("writing"). Historically defined as the art and science of constructing secret codes to keep messages private.

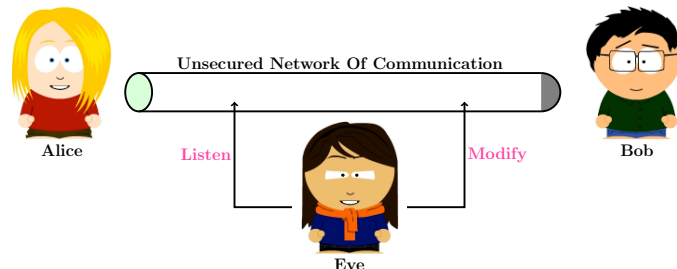
Cryptanalysis: The study and mathematical analysis of secret codes with the aim of breaking them or deciphering hidden information without prior knowledge of the secret key.

While these are classical definitions, as we go on with the course, we will see what modern cryptography and cryptanalysis are. In our context, cryptology is the scientific study of the protection of information.

1.1 Why Cryptography?

But why we need cryptography? One of the most common use is secure communication. Consider the model:

- **Alice** and **Bob** wish to communicate over an unsecured network (e.g., the public Internet).
- **Eve** (an adversary or eavesdropper) controls or observes the channel, attempting to intercept, alter, or forge messages.



The goal is to find **Efficient** algorithms with the following properties:

- **Confidentiality:** Ensures that only the intended recipient (Bob) can read Alice's message.
- **Integrity:** Guarantees that the message was not modified, tampered with, or corrupted without detection.
- **Authenticity:** Provides proof of origin, verifying that the message genuinely originated from Alice and not an impersonator.

2 Applications & Career Relevance

Cryptography is not merely theoretical; it provides the core primitives underlying modern digital security.

- In terms of academic research, Cryptology is a dynamic, evolving discipline. Emerging paradigms require novel mathematical techniques to handle novel threat models. So a lot of open directions of research are available.
- Industry leaders (e.g., Google, Microsoft, Apple) rely heavily on cryptographic specialists to design secure protocols, safeguard user data, and architect enterprise cloud security.
- Companies such as Intel, AMD, and Qualcomm embed cryptographic operations directly into hardware circuits (e.g., AES-NI extensions, secure execution environments, and hardware roots of trust).
- Whether conducting code audits, penetration testing, or protocol engineering, a strong grounding in formal cryptology is essential for identifying subtle implementation vulnerabilities.

3 Classical Ciphers

Before studying modern symmetric and asymmetric ciphers, we examine historical **classical ciphers**. Evidence of secret writing appears as early as 1900 BCE in ancient Egyptian tomb inscriptions. One of the earliest systematic ciphers is the **Caesar Cipher**, utilized around 50 BCE by Julius Caesar.

3.1 The Caesar Cipher

The Caesar Cipher is an elementary **monoalphabetic substitution cipher**. Each character in the plaintext is replaced by a character situated a fixed number of positions further down the alphabet. In Caesar's traditional setup, the shift value was fixed at 3:

- **Alphabet Mapping:** Let $\mathbb{Z}_{26} = \{0, 1, 2, \dots, 25\}$ represent the alphabet:

$$A \rightarrow 0, B \rightarrow 1, \dots, Z \rightarrow 25.$$

- **Encryption Function:** $E(x) = (x + 3) \pmod{26}$
- **Decryption Function:** $D(x) = (x - 3) \pmod{26}$
- **Example:** hello $\rightarrow$ **KHOOR**, world $\rightarrow$ **ZRUOG**

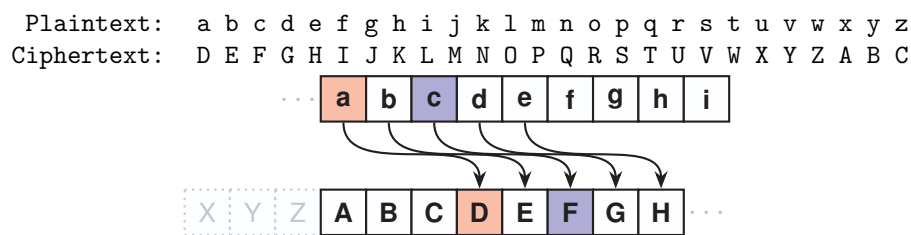


Table 1: The Caesar Cipher

The primary vulnerability of the traditional Caesar cipher is that secrecy relies entirely on the secrecy of the *algorithm itself*. Anyone who knows how it is encrypted can decrypt any message.

3.2 The Shift Cipher (Caesar with Keys)

Historical accounts suggest variations existed for Caesar's cipher, for instance, Augustus Caesar reportedly used a shift of 1. But, can you see the pattern? Here comes the concept of a cryptographic key (**K**), which generalizes the shift value.

- **Key:** $K \in \mathcal{K} = \{0, 1, 2, \dots, 25\}$
- **Encryption:** For any plaintext symbol $x \in \mathcal{M}$: $e_K(x) = (x + K) \pmod{26}$
- **Decryption:** For any ciphertext symbol $y \in \mathcal{C}$: $d_K(y) = (y - K) \pmod{26}$

While introducing a key is a step forward, the key space contains only **26 possible keys**. An attacker can easily test all 26 keys by hand in a matter of minutes. This is vulnerable to a *brute-force key search attack*.

3.3 Monoalphabetic Substitution Cipher

In the shift cipher, encryption is performed by shifting (rotating) each letter of the plaintext by a fixed number of positions. From a mathematical perspective, this operation is simply a permutation of the alphabet. But why restrict ourselves to only this particular type of permutation? Instead, we can choose any arbitrary permutation. This idea leads to the Permutation (Substitution) Cipher, where each plaintext

letter is replaced according to a secret permutation of the alphabet. The total number of possible keys is $26!$, making the key space enormously larger than that of the Caesar cipher. Such substitution ciphers were widely used until the end of the 19th century.

Here is an example of such secret permutation given in Table 2. Using this: cryptographicprotocol $\rightarrow$ ZYONJBEYQNDZCZNYBJBZBR.

Plaintext:	a b c d e f g h i j k l m n o p q r s t u v w x y z
Ciphertext:	Q A Z X S W E D C V F R T G B N H Y U J M K I L O P

Table 2: A Monoalphabetic Substitution Cipher

Mathematical Formulation: Let $\pi \in S_{26}$ be a symmetric group permutation mapping the set $\{A, \dots, Z\}$ to itself.

- **Key:** The permutation rule π .
- **Key Space Size:** The total number of unique permutations is $26!$ (26 factorial):

$$|\mathcal{K}| = 26! \approx 4.03 \times 10^{26} \approx 2^{88.3}$$

- **Encryption:** $e_{\pi}(x) = \pi(x)$
- **Decryption:** $d_{\pi}(y) = \pi^{-1}(y)$

With an effective key size of approximately **88 bits**, an exhaustive brute-force search over all $26!$ candidate permutations is computationally infeasible. Despite its vast key space, the general monoalphabetic substitution cipher is insecure against a **Ciphertext-Only Attack** using **Frequency Analysis**.

Simple substitution ciphers maintain a 1-to-1 mapping between plaintext letters and ciphertext letters. Consequently, the cipher **preserves the statistical structure** of the underlying natural language. For example, in standard English language, character distributions are non-uniform. The letter **E** appears with the highest average frequency ($\approx 12.7\%$), followed by **T** ($\approx 9.1\%$), **A** ($\approx 8.2\%$), and **O** ($\approx 7.5\%$) as shown in the graph in Figure 1.

An attacker can collect ciphertext character frequency counts, compare them to standard English relative frequency tables, and exploit common letter pairs (*bigrams*, e.g., “TH”, “HE”) or triplets (*trigrams*, e.g., “THE”) to systematically reconstruct π^{-1} .

Assignment: Character Frequency Analyzer

Objective: Implement a C/C++ tool to perform character frequency analysis on an arbitrary text corpus. Read an unparsed text file (e.g., raw text extracted from Wikipedia) and count the occurrences of each letter ($A-Z$) and compute its relative frequency percentage:

$$f(c) = \frac{\text{Count}(c)}{\text{Total Alphabetic Characters}} \times 100\%$$

Note that you have to handle both the uppercase and lowercase characters before tabulating. Inspect the standard ASCII table using the terminal command and pay close attention to character codes for upper ('A' = 65, 'Z' = 90) and lower ('a' = 97, 'z' = 122) ranges.

3.4 Vigenère Cipher

Here we use different shift values for different letters and the keywords of a specific length ℓ determines the shift values: $a \rightarrow 0$, $b \rightarrow 1$, $\dots$, $z \rightarrow 25$.. The Vigenère Cipher is a polyalphabetic substitution cipher and the keyword is **repeated** periodically until it matches the length of the plaintext. This cipher was considered secure until the end of the 19th century. An example is given in Table 3.

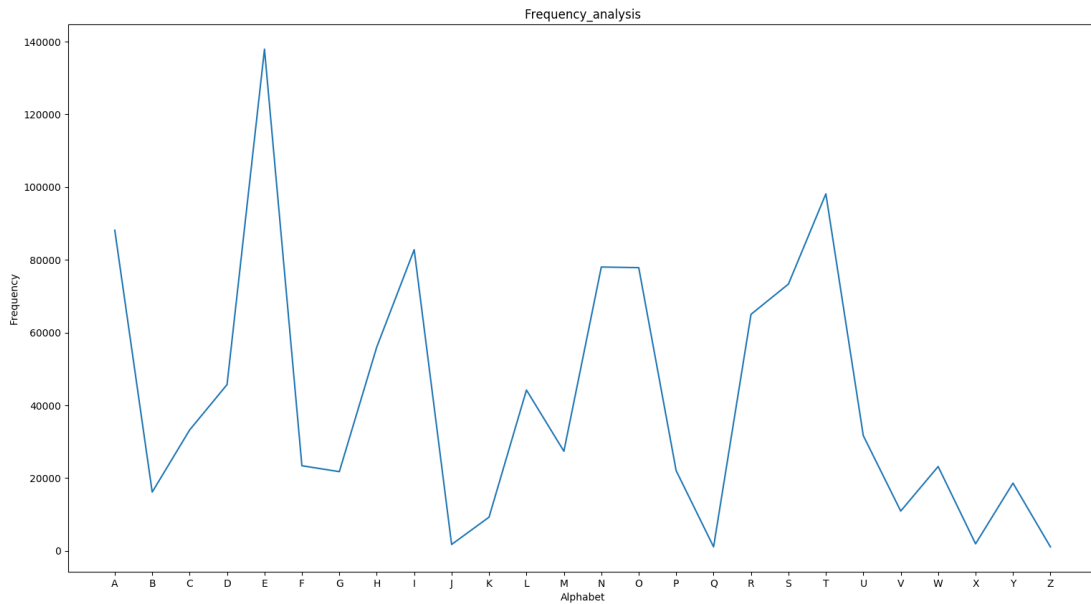


Figure 1: The distribution of letter frequencies in standard English language

Plaintext	s	e	c	r	e	t	m	e	s	s	a	g	i	n	g
Keyword	c	o	d	e	c	o	d	e	c	o	d	e	c	o	d
Shift	2	14	3	4	2	14	3	4	2	14	3	4	2	14	3
Ciphertext	U	S	F	V	G	H	P	I	U	G	D	K	K	B	J

Table 3: Example of Vigenère Cipher

4 Breaking the Vigenère Cipher using Kasiski's Method

This tutorial is to help you understand the classical attack against the Vigenère cipher. The classical attack against the Vigenère cipher is known as **Kasiski's Method**. The attack consists of two independent stages.

1. Estimate the length of the secret key.
2. Recover each letter of the secret key individually.

Once the secret key has been recovered, decrypting the ciphertext becomes straightforward. Suppose the secret key is MATH, whose length is $\ell = 4$. During encryption, the key is repeated. For example,

Plaintext	A	T	T	A	C	K	A	T	D	A	W	N
Key	M	A	T	H	M	A	T	H	M	A	T	H

Notice that

- positions 0, 4, 8, ... are encrypted using the letter M,
- positions 1, 5, 9, ... are encrypted using the letter A,
- positions 2, 6, 10, ... are encrypted using the letter T,
- positions 3, 7, 11, ... are encrypted using the letter H.

Therefore, if the key length is known, the ciphertext can be divided into ℓ independent groups. For $\ell = 4$, we obtain four slices or buckets

$$\begin{aligned} S_1 &: C_0, C_4, C_8, C_{12}, \dots \\ S_2 &: C_1, C_5, C_9, C_{13}, \dots \\ S_3 &: C_2, C_6, C_{10}, C_{14}, \dots \\ S_4 &: C_3, C_7, C_{11}, C_{15}, \dots \end{aligned}$$

Each group has been encrypted using exactly one shift. Hence,

Breaking one Vigenère cipher is equivalent to breaking ℓ independent shift ciphers.

4.1 Step 1: Estimating the Key Length

The first challenge is determining the unknown key length. To do this we use the *Index of Coincidence* (*IoC*). Suppose a text contains N letters. Let n_i denote the number of occurrences of the i -th letter of the alphabet. The *Index of Coincidence* is defined as

$$I = \frac{\sum_{i=0}^{25} n_i(n_i - 1)}{N(N - 1)}.$$

The IoC measures the probability that two randomly selected letters from the text are identical. Typical values are

$$\begin{aligned} \text{Random text} &\approx 1, \\ \text{English text} &\approx 1.7. \end{aligned}$$

4.1.1 Estimating the Key Length Using IoC

Assume the candidate key length is ℓ . Split the ciphertext into ℓ slices. For example, if $\ell = 4$,

$$\begin{aligned} S_1 &: C_0, C_4, C_8, C_{12}, \dots \\ S_2 &: C_1, C_5, C_9, C_{13}, \dots \\ S_3 &: C_2, C_6, C_{10}, C_{14}, \dots \\ S_4 &: C_3, C_7, C_{11}, C_{15}, \dots \end{aligned}$$

If the guessed key length is correct, each slice has been encrypted using a single shift cipher and therefore resembles ordinary English. Consequently, its IoC will be close to that of English. If the guessed key length is incorrect, each slice mixes several shift ciphers together, producing text that is closer to random. Consequently, its IoC will be much smaller. Therefore, we compute the average IoC for every candidate key length and select the smallest candidate whose average IoC is close to that of English.

4.1.2 Algorithm for Estimating the Key Length

Input: Ciphertext C , maximum key length L .

Output: Estimated key length ℓ .

1. For every candidate key length $j = 1, 2, \dots, L$:
 - (a) Split the ciphertext into j slices.
 - (b) Compute the IoC of every slice.
 - (c) Compute the average IoC.
2. Return the smallest key length whose average IoC is closest to English.

4.2 Step 2: Recovering Each Key Letter Using Frequency Analysis

Once the key length ℓ has been determined, the ciphertext is divided into ℓ independent slices. Since all letters in a slice are encrypted using the same shift, each slice is simply a shift cipher. The goal is therefore to determine the shift for each slice. Suppose S is one ciphertext slice. Since a Caesar cipher has only 26 possible shifts, we simply try every possible shift. For every candidate shift $k \in \{0, 1, \dots, 25\}$, we perform the following steps.

1. Decrypt the slice using shift k .
2. Compute the letter-frequency vector $\gamma^k = (\gamma_0^k, \gamma_1^k, \dots, \gamma_{25}^k)$, where γ_i^k denotes the relative frequency of the i -th letter in the decrypted slice.
3. Let $\delta = (\delta_0, \delta_1, \dots, \delta_{25})$ denote the standard frequency distribution of English text.
4. Compare the two frequency vectors using the *cosine similarity*

$$\cos(\theta) = \frac{\gamma^k \cdot \delta}{\|\gamma^k\| \|\delta\|}, \text{ where } \gamma^k \cdot \delta = \sum_{i=0}^{25} \gamma_i^k \delta_i.$$

The cosine similarity measures how closely the decrypted slice resembles ordinary English. If the value of cosine similarity is close to 1 (say more than 0.9), it indicates that the decrypted slice has a letter distribution closer to English and is therefore more likely to have been decrypted using the correct Caesar shift.

Assignment

This is the ciphertext of a Vigenère cipher:

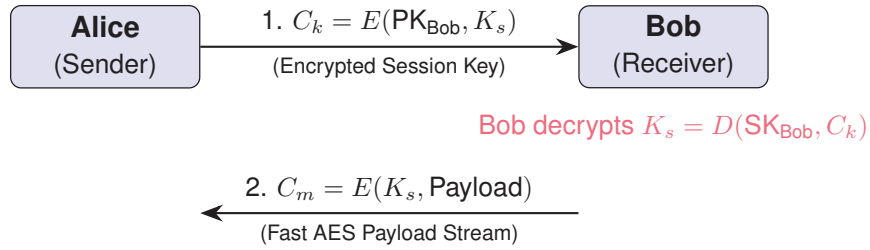
```
DVNSJ QRVPC GWBVZ QGVVP VILQJ VTURP BNPZN TGQIG SIGVW MTCIM FQUIA CCKVG CBYKQ
ZNTNB UCIEN UHIVF IWPQC BNKCM IGGGV OEWEV PVGFX APQKM EATDR TBIQG IPRNX JECGQ
NPLMY EDURF IPRXX AVVDZ JKIPN UBQYG PVQGM XYCXV RFIPN VIPRI GMNVT AGVGM NUJZR
KCBUG QCVNS QAILI FPDBN UXVTN TJBQZ JHVIP RMCWJ NTLTG IPNVR WAPTK GGSBU GBIYN
TDRTN ZBQBK BPIIV PTLZC CCFEG QCVHW AFXNS GGMAV HCOLT KGUXV PNJLV PVUNV WMZCI
QPUPA GTDVB ONXUK AWFQE PLOTL VEXVR GCOVP TMEKC ONPSP VUIWE AIPRU RPNBP ZQGRQ
QGSBB UEMAF IPRFP GRZET BTXVT GKMEA RWEPT ZBHIP RNXJE CGGGJ TNVTH BEQDU PQCBN
KCMQD DWXUD VZCIP ROPBV EHBUG HKUQA IETTI QCQWH VCCZD TZFIT WZGIZ VEUQT WGMFC
CLRNT ONPIX EQDNF VWIGJ PLVPH XVTTL TGCME CIQBP HWSVW QAMTZ FQCMZ CCCFE GQCVS
MFEGQ OGSPB YHQZR AMBDH MEXPB VQCAP QJTQN TIQVD LRGE B UGDZR OHEUK AMNPD BUGGM
KRAIV PTLUQ LIOUI ZNEIQ QGPAB HIMAH DCAFE ZNEIQ PCAIC RAQPC IQBPH KRPIC EKTAY
CIMEV WMFEW WYCGZ RCAQF GSBUC IUNVW MZCIQ PULIF PDBZG GMYAP KBNAM PVXWA QUNBT
BCYCH JHVPT NPVCN ITNBT SMFEG QOKCO CCIBR TCANP SZRCH WAKCO NDDCG VWMJQ GTQ
```

Your task is to write a **Python** program that breaks the Vigenère cipher and finds the encryption key. The program must decrypt the ciphertext using the Vigenère cipher decryption algorithm (i.e., you have to implement a decryption function) and output the plaintext.

5 Symmetric and Asymmetric Key Cryptography

Previously, we discussed several classical ciphers. Although these ciphers are useful for understanding the history of cryptography and the basic principles of encryption, they are no longer used in practice.

The reason is that modern communication systems require two fundamental properties: **efficiency** and **security**. Encryption and decryption should be fast enough to process large amounts of data, while at the same time providing strong security against computationally powerful adversaries. Classical ciphers fail to satisfy these requirements. Therefore, from this point onward, we focus on *modern cryptography*.



Modern cryptosystems can be broadly classified into two categories: *symmetric-key cryptography* and *asymmetric-key (public-key) cryptography*.

A *symmetric-key cryptosystem* uses the same secret key for both encryption and decryption. Consequently, the sender and the receiver must share this secret key before communication begins. Since anyone possessing the key can decrypt the ciphertext, the secrecy of the key is of utmost importance. Symmetric-key algorithms are computationally very efficient and are therefore ideal for encrypting large amounts of data, such as hard drives (e.g., BitLocker), files, databases, VPN traffic, and network communication. Their main limitation, however, is the *key distribution problem*: both communicating parties must somehow establish the shared secret key over an insecure network.

An *asymmetric-key cryptosystem* or *public-key cryptosystem* addresses this key distribution problem by using two mathematically related keys: a *public key*, which can be freely distributed, and a *private key*, which is known only to its owner. Anyone can encrypt a message using the public key, but only the holder of the corresponding private key can decrypt it. This makes public-key cryptography particularly suitable for establishing secure communication over untrusted networks, authenticating users, and creating digital signatures. The disadvantage is that public-key operations involve expensive mathematical computations and are typically hundreds to thousands of times slower than symmetric-key algorithms.

In practice, modern security protocols combine the strengths of both approaches using a *hybrid* strategy. During the initial phase of communication, asymmetric cryptography is used to authenticate the communicating parties and securely exchange a short-lived *session key*. Once this shared session key has been established, all subsequent communication is encrypted using a fast symmetric-key algorithm such as AES. In this way, both the convenience of public-key cryptography and the efficiency of symmetric-key cryptography are combined. Nearly all modern secure communication protocols, including HTTPS/TLS, SSH, and Signal, follow this approach.

6 Symmetric Key Cryptosystem

Having introduced the basic idea of symmetric-key cryptography, we now give its formal mathematical definition. Informally, a symmetric-key cryptosystem consists of three algorithms: a *key generation algorithm*, an *encryption algorithm*, and a *decryption algorithm*. These algorithms operate over a plaintext space, a ciphertext space, and a finite set of secret keys.

Formal Definition: Symmetric Key Cryptosystem (Cipher)

A symmetric-key cryptosystem is a five-tuple $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$, where the following conditions are satisfied:

- $\mathcal{P}$ is a finite set of possible plaintexts;
- $\mathcal{C}$ is a finite set of possible ciphertexts;
- $\mathcal{K}$, the key space, is a finite set of possible keys;
- $\mathcal{E}$ and $\mathcal{D}$ are set of deterministic and **publicly known** algorithms

Such that for each $K \in \mathcal{K}$, there is an encryption rule $e_K \in \mathcal{E}$ and a corresponding decryption rule $d_K \in \mathcal{D}$. Each $e_K : \mathcal{P} \rightarrow \mathcal{C}$ and $d_K : \mathcal{C} \rightarrow \mathcal{P}$ are functions such that $d_K(e_K(P)) = P$ for every plaintext element $P \in \mathcal{P}$. A schematic representation of a symmetric-key cryptosystem is shown in Figure 2.

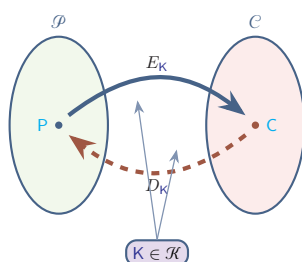


Figure 2: Symmetric-key cryptosystem.

Kerckhoffs' Principle. Notice that in the above definition we explicitly require the encryption and decryption algorithms to be *publicly known*. This reflects one of the most fundamental principles in modern cryptography, known as *Kerckhoffs' Principle*:

A cryptographic system should remain secure even if everything about the system is public knowledge except the secret key.

In other words, we assume that an adversary knows the complete design of the cipher, including its algorithms, implementation details, and even the source code. The only information that must remain secret is the cryptographic key K . Designing a cipher whose security relies on hiding the algorithm itself is known as *security through obscurity*, and is considered poor cryptographic practice.

6.1 Cipher Design Criteria

A symmetric-key cryptosystem should satisfy two fundamental requirements: *efficiency* and *security*.

First, encryption and decryption should be computationally efficient whenever the correct secret key is available. What constitutes an efficient implementation depends on the target platform. For example, a cipher that performs well in software may require a large amount of hardware resources, while another cipher may be optimized for lightweight hardware but achieve lower software performance.

Second, without knowledge of the secret key, recovering the plaintext from the ciphertext should be computationally infeasible. Likewise, it should be computationally infeasible to recover the secret key from any collection of observed ciphertexts. A well-designed cipher should also conceal all meaningful structural information about the underlying plaintext so that an adversary gains no useful information from observing the ciphertext alone.

Throughout this course we will make these notions of security precise by introducing formal security models and rigorous mathematical definitions.